GenAI Unit 3 Hands-on 1

Name: K L Sonika **SRN:** PES2UG23CS247

Section: D **Date:** 03-04-2026

Advanced RAG and Retrieval

```
c:\Users\klson\AppData\Local\Programs\Python\Python313\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets
  from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 7084.39it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
=== Dense Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=0.7550: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.3656: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_1] score=0.2918: BERT is a bidirectional encoder trained using masked language modellin

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=0.5714: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_0] score=0.1448: Transformers use self-attention mechanisms to process sequences in par
  Rank 3 [doc_4] score=0.1129: Neural networks learn by adjusting weights through backpropagation.
```

Observation

- For Query 1 (semantic), dense retrieval does well — it understands "neural nets learn" ≈ "backpropagation / weights".
- For Query 2 (keyword exact-match), dense retrieval may **not** rank `doc_2` (the BM25 document) at the top — the embedding model sees "BM25 term frequency" as a technical phrase it doesn't strongly embed near the correct document.

This is the core gap. Let's now see the other side of the coin: **Sparse Retrieval**.

```
=== BM25 Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=1.2259: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_2] score=0.0000: The BM25 algorithm ranks documents based on term frequency and inverse

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=2.9625: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_4] score=0.0000: Neural networks learn by adjusting weights through backpropagation.
  Rank 3 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
```

Observation

- BM25 excels at Query 2 (exact keyword match) — it immediately finds `doc_2` with the highest score because the terms "BM25" and "term" and "frequency" all appear in it.
- BM25 **fails** at Query 1 (semantic) — it scores 0 for documents that don't share exact words with the query ("neural nets learn" has no token overlap with "backpropagation" or "weights").

Neither BM25 nor Dense alone is sufficient. We need both.

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 10897.87it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
=== ColBERT-style Late Interaction Scores ===
Query: 'How do neural nets learn?'

  doc_0 score=1.8954: Transformers use self-attention mechanisms to process sequences in par
  doc_1 score=2.1215: BERT is a bidirectional encoder trained using masked language modellin
  doc_2 score=1.8315: The BM25 algorithm ranks documents based on term frequency and inverse
  doc_3 score=1.9923: Gradient descent is an optimization technique used to minimize the los
  doc_4 score=3.3560: Neural networks learn by adjusting weights through backpropagation.

Top document: doc_4 — 'Neural networks learn by adjusting weights through backpropagation.'
```

Key Difference Summary

Model	Representation	Speed	Quality
BM25	Sparse (bag-of-words)	Very fast	Great for exact keywords
SBERT	1 dense vector per sentence	Fast (pre-encoded)	Great for semantic meaning
ColBERT	N dense vectors (1 per token)	Slower, more memory	Best of both — token-level semantics

```
STEP 3: Reciprocal Rank Fusion (k=60)
--------------------------------------------------------------------------------
Doc      BM25 rank    SBERT rank    RRF(BM25)      RRF(SBERT)     Total RRF
--------------------------------------------------------------------------------
  doc_0  5            4             0.015385       0.015625       0.031010
  doc_1  4            3             0.015625       0.015873       0.031498
  doc_2  3            5             0.015873       0.015385       0.031258
  doc_3  2            2             0.016129       0.016129       0.032258
  doc_4  1            1             0.016393       0.016393       0.032787

================================================================================
FINAL HYBRID RANKING
================================================================================
  #1  doc_4 (RRF=0.032787): Neural networks learn by adjusting weights through backpropagation.
  #2  doc_3 (RRF=0.032258): Gradient descent is an optimization technique used to minimize the loss function.
  #3  doc_1 (RRF=0.031498): BERT is a bidirectional encoder trained using masked language modelling.
  #4  doc_2 (RRF=0.031258): The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #5  doc_0 (RRF=0.031010): Transformers use self-attention mechanisms to process sequences in parallel.
```

Interpreting the Numbers

Let's trace `doc_4` ("Neural networks learn by adjusting weights through backpropagation."):

- **BM25 rank**: Likely 1 or 2 (tokens "neural", "learn" match the query exactly)
- **SBERT rank**: Also likely 1 or 2 (semantically, this is the closest sentence)

RRF reward = `1/(60+1) + 1/(60+1)` = `0.01639 + 0.01639` = `0.03279`

A document that ranks **#1 in both lists** gets the maximum possible RRF score. A document that ranks in the middle of one but bottom of the other still gets partial credit — it won't be buried.

The beauty of RRF: It doesn't matter if BM25 gives scores of `4.7` and SBERT gives `0.8`. We only care about *positions in the ranked list*.

```
COMPARISON: BM25 vs SBERT vs Hybrid (for query: 'How do neural nets learn?')
==========================================================================================
Rank   BM25 top doc                                        SBERT top doc
------------------------------------------------------------------------------------------
  #1   Neural networks learn by adjusting weights th       Neural networks learn by adjusting weights th
  #2   Gradient descent is an optimization technique       Gradient descent is an optimization technique
  #3   The BM25 algorithm ranks documents based on t       BERT is a bidirectional encoder trained using
  #4   BERT is a bidirectional encoder trained using       Transformers use self-attention mechanisms to
  #5   Transformers use self-attention mechanisms to       The BM25 algorithm ranks documents based on t

Hybrid Final Ranking:
  #1  Neural networks learn by adjusting weights through backpropagation.
  #2  Gradient descent is an optimization technique used to minimize the loss function.
  #3  BERT is a bidirectional encoder trained using masked language modelling.
  #4  The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #5  Transformers use self-attention mechanisms to process sequences in parallel.
```

```
Query: 'BM25 term frequency ranking'

Doc      BM25 rank    SBERT rank    RRF Score    Document
--------------------------------------------------------------------------------
  doc_0  5            5             0.030769   Transformers use self-attention mechanisms to process sequen
  doc_1  4            3             0.031498   BERT is a bidirectional encoder trained using masked languag
  doc_2  1            1             0.032787   The BM25 algorithm ranks documents based on term frequency a
  doc_3  3            4             0.031498   Gradient descent is an optimization technique used to minimi
  doc_4  2            2             0.032258   Neural networks learn by adjusting weights through backpropa

Hybrid Final Ranking:
  #1  doc_2: The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #2  doc_4: Neural networks learn by adjusting weights through backpropagation.
  #3  doc_1: BERT is a bidirectional encoder trained using masked language modelling.
  #4  doc_3: Gradient descent is an optimization technique used to minimize the loss function.
  #5  doc_0: Transformers use self-attention mechanisms to process sequences in parallel.
```

Why `k=60` Specifically?

The constant `k=60` was empirically determined in the original RRF paper (Cormack et al., 2009) to work well across many retrieval benchmarks. It prevents the #1 rank from dominating too heavily.

k value	Effect
k=0	Rank 1 gets ∞ weight — effectively winner-takes-all
k=1	Rank 1: 0.5, Rank 2: 0.33 — very steep drop-off
k=60	Rank 1: 0.0164, Rank 2: 0.0161 — gentle, democratic
k=1000	All ranks are near-equal — no differentiation

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 10623.45it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Query: 'How do neural nets learn?'
  → doc_4 (RRF=0.03279): Neural networks learn by adjusting weights through backpropagation.
  → doc_3 (RRF=0.03226): Gradient descent is an optimization technique used to minimize the loss function.

Query: 'BM25 term frequency ranking'
  → doc_2 (RRF=0.03279): The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  → doc_4 (RRF=0.03226): Neural networks learn by adjusting weights through backpropagation.

Query: 'attention in transformers'
  → doc_0 (RRF=0.03279): Transformers use self-attention mechanisms to process sequences in parallel.
  → doc_4 (RRF=0.03226): Neural networks learn by adjusting weights through backpropagation.
```

Summary

Technique	Strength	Weakness	Library
BM25	Exact keywords, proper nouns, rare terms	No semantic understanding	`rank_bm25`
SBERT	Semantic similarity, paraphrases	Misses exact keyword matches	`sentence-transformers`
ColBERT	Token-level semantics, best quality	Slower, higher memory	`colbert-ai/colbert`
Hybrid (RRF)	Best of both worlds, scale-agnostic fusion	More compute than either alone	Custom (built above)

Key Takeaways

1. **Sparse retrieval** (BM25) = bag-of-words, term statistics, exact match power.
2. **Dense retrieval** (SBERT/ColBERT) = neural embeddings, semantic understanding.
3. **Hybrid retrieval** (BM25 + SBERT + RRF) = combines both by fusing rank lists, not raw scores.
4. **RRF formula**: `score(d) = Σ 1/(k + rank(d))` — simple, powerful, no hyperparameter tuning needed.

Next: Notebook 2 will improve the retrieved results further using **Re-Ranking** and **Query Expansion** before they reach the LLM.

Reranking and Query Expansion

```
STAGE 1 — Bi-Encoder Top-5 Candidates
----------------------------------------------------------------------
  #1  [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.
  #4  [score=0.3478] Transformers use self-attention mechanisms to process sequences in parallel.
  #5  [score=0.3418] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
STAGE 2 — Cross-Encoder Re-Ranking
----------------------------------------------------------------------
  #1  [CE score=3.3522] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [CE score=-4.4683] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [CE score=-5.4407] Transformers use self-attention mechanisms to process sequences in parallel.
  #4  [CE score=-6.9237] BERT is a bidirectional encoder trained using masked language modelling.
  #5  [CE score=-10.4939] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.

--- Before vs After Re-Ranking ---
Bi-Encoder Rank 1:            The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
Cross-Encoder Rank 1:         The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
```

What Just Happened?

The cross-encoder re-reads each (query, document) pair as a single combined input. This means:

- "attention" in the query can cross-attend to "attention mechanism" in the document at the token level.
- The model outputs a single relevance score (a logit — can be negative or large positive).

Rule of thumb: Use bi-encoders when your corpus has >10,000 documents (too slow to cross-encode all). Use the cross-encoder only on the top-20/50 returned by the bi-encoder.

```
Original Query:
  'How does attention work in language models?'

Hypothetical Document (generated by Gemini):
  Attention mechanisms in language models allow the model to focus on the most relevant parts of the input sequence when processing each word. This is achieved by assigning weights to
```

```
Query: 'How does attention work in language models?'

Raw Query Retrieval (Top 3):
  #1 [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.

HyDE Retrieval (Top 3):
  #1 [score=0.5635] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.5432] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.4569] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
Generated query variants:
  (original): How does attention work in language models?
  (variant 1): Can you explain the attention mechanism used in language models?
  (variant 2): What are the key components and processes involved in attention within language models?
  (variant 3): In what ways does attention enhance the performance of language models?

Multi-Query retrieved 5 unique documents (union of all variants):
  [doc_8] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  [doc_9] Large language models are trained on massive text corpora to learn general-purpose representations.
  [doc_1] BERT is a bidirectional encoder trained using masked language modelling.
  [doc_0] Transformers use self-attention mechanisms to process sequences in parallel.
  [doc_6] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

HyDE vs Multi-Query: When to Use Which?

Technique	Best For	Weakness
HyDE	Short, vague queries; when query language ≠ document language	LLM hallucination in the hypothetical doc can mislead retrieval
Multi-Query	Covering multiple angles of a question; exploratory search	Higher latency (multiple LLM calls); may retrieve noisy documents

```
Query: 'How does attention work in language models?'
======================================================================
[1] HyDE Expansion: Attention mechanisms in language models allow the model to focus on the most relevant parts of the input sequence when processing each word. This is a...

[2] Hybrid Retrieval (5 candidates):
     #1: The attention mechanism computes a weighted sum of value vectors based on query-
     #2: Large language models are trained on massive text corpora to learn general-purpo
     #3: The BM25 algorithm ranks documents based on term frequency and inverse document
     #4: Transformers use self-attention mechanisms to process sequences in parallel.
     #5: Neural networks learn by adjusting weights through backpropagation.

[3] After Re-Ranking (top 3):
     #1: The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
     #2: Large language models are trained on massive text corpora to learn general-purpose representations.
     #3: Transformers use self-attention mechanisms to process sequences in parallel.

[4] Final Answer:
     Attention computes a weighted sum of value vectors based on query-key similarity (Document 1). Transformers use self-attention mechanisms to process sequences in parallel (Document
```

```
'Attention computes a weighted sum of value vectors based on query-key similarity (Document 1). Transformers use self-attention mechanisms to process sequences in parallel (Document 3)
```

```
Query: 'What is the purpose of backpropagation?'
====================================================================
[1] HyDE Expansion: Backpropagation, short for "backward propagation of errors," is a supervised learning algorithm used to train artificial neural networks. Its primary ...

[2] Hybrid Retrieval (5 candidates):
     #1: Gradient descent is an optimization technique used to minimize the loss function
     #2: Neural networks learn by adjusting weights through backpropagation.
     #3: The attention mechanism computes a weighted sum of value vectors based on query-
     #4: The BM25 algorithm ranks documents based on term frequency and inverse document
     #5: Fine-tuning adapts a pre-trained model to a specific downstream task using task-

[3] After Re-Ranking (top 3):
     #1: Neural networks learn by adjusting weights through backpropagation.
     #2: Gradient descent is an optimization technique used to minimize the loss function.
     #3: The attention mechanism computes a weighted sum of value vectors based on query-key similarity.

[4] Final Answer:
     Neural networks learn by adjusting weights through backpropagation.

'Neural networks learn by adjusting weights through backpropagation.'
```

Summary

The Full Advanced RAG Stack

```
User Query
    |
    ▼
Query Expansion (HyDE / Multi-Query)
    |  → Makes the query richer and closer to document language
    ▼
Hybrid Retrieval (BM25 + SBERT + RRF)
    |  → Combines keyword and semantic search
    ▼
Cross-Encoder Re-Ranking
    |  → Selects the truly relevant documents from candidates
    ▼
LLM Generation (Gemini)
    |  → Produces a grounded, accurate answer
    ▼
Answer
```

Comparison of Approaches

Stage	Naïve RAG	Advanced RAG
Query	Raw user input	HyDE expanded / multi-phrasing
Retrieval	Dense only (cosine)	Hybrid (BM25 + SBERT + RRF)
Candidate filtering	Top-K by embedding score	Cross-encoder re-ranking
Generation	All top-K in context	Only highest-quality docs

Next: Notebook 3 will shift from the retrieval side to the **generation side** — fine-tuning, quantization, and Mixture of Experts.

Generation Enhancement

```
[9]  2.7s

...  Original weight W:      64x64 = 4,096 parameters
     LoRA matrices A+B:      4x64 + 64x4 = 512 parameters
     LoRA overhead:          12.50% of original

     Original output:  [ 0.0417  -0.05841  0.07145  0.00424]
     LoRA output:      [ 0.0417  -0.05841  0.07145  0.00424]
     (outputs identical at init because B=0, so ΔW = B@A = 0)
```

```
...  c:\Users\klson\AppData\Local\Programs\Python\Python313\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets
       from .autonotebook import tqdm as notebook_tqdm
     Loading a tiny model for PEFT demo (distilgpt2)...
     Loading weights: 100%|██████████| 76/76 [00:00<00:00, 3797.11it/s]
     trainable params: 147,456 || all params: 82,060,032 || trainable%: 0.1797
     c:\Users\klson\AppData\Local\Programs\Python\Python313\Lib\site-packages\peft\tuners\lora\layer.py:2285: UserWarning: fan_in_fan_out is set to False but the target module is `Conv1D`. S
       warnings.warn(

     Generate   + Code   + Markdown
```

```
...  Original value:  1.3456789
     FP32:            1.3456789255  (error: 2.55e-08)
     FP16:            1.3457031250  (error: 2.42e-05)
     BF16:            1.3437500000  (error: 1.93e-03)

     --- Memory footprint of a 1-billion parameter model ---
       FP32        : 3.73 GB
       FP16/BF16   : 1.86 GB
       INT8        : 0.93 GB
       INT4        : 0.47 GB
```

```
...  FP16 overflow example:
       Value:   70000.0
       FP32:    70000.0  (correct)
       FP16:    inf  ← OVERFLOW! Becomes inf
       BF16:    70144.0  (safe — wide exponent range)
```

```
...  Linear Quantization Demo (FP32 → INT8 → FP32)
     ================================================================
     Scale factor S = 0.007111
     Zero point   Z = -95

     Index    FP32         INT8       Reconstructed    Error
     ----------------------------------------------------------------
       [0]     0.496714     -25         0.497787   0.001073
       [1]    -0.138264    -114        -0.135114   0.003151
       [2]     0.647689      -4         0.647123   0.000566
       [3]     1.523030     119         1.521805   0.001225
       [4]    -0.234153    -128        -0.234671   0.000518
       [5]    -0.234137    -128        -0.234671   0.000534
       [6]     1.579213     127         1.578695   0.000518
       [7]     0.767435      13         0.768014   0.000579

     Max quantization error: 0.003151
     Mean quantization error: 0.001020

     Memory: FP32=32 bytes → INT8=8 bytes (4.0x reduction)
```

```
...  Model Size Comparison for Common LLMs
     ================================================================
     Model               Params       FP32 (GB)    FP16 (GB)    INT8 (GB)    INT4 (GB)
     ----------------------------------------------------------------
     distilgpt2          0.07B       0.25         0.12         0.06         0.03
     GPT-2               0.12B       0.44         0.22         0.11         0.05
     BERT-Base           0.11B       0.41         0.20         0.10         0.05
     Llama-3.2-1B        1.00B       3.73         1.86         0.93         0.47
     Llama-3.1-8B        8.00B       29.80        14.90        7.45         3.73
     Llama-3.1-70B       70.00B       260.77       130.39       65.19        32.60
```

```
Quantization Trade-off (100 weights)
Bits     MSE (error)        Size (bytes)    Compression
-------------------------------------------------------
  32       0.23258863          400.0            1.0x
  16       0.00000000          200.0            2.0x
  8        0.00003263          100.0            4.0x
  4        0.00870766          50.0             8.0x
  2        0.19430648          25.0             16.0x
C:\Users\klson\AppData\Local\Temp\ipykernel_14872\702760880.py:16: RuntimeWarning: invalid value encountered in cast
```

```
🔢 Expert: MATH
Query:  What is the integral of sin(x)dx?
Answer: To find the integral of $\sin(x)$, we can use the following formula:

$$\int \sin(x) \, dx = -\cos(x) + C$$

where $C$ is the constant of integration.

**Proof:**

We can prove this using the following steps:

1. Recall the derivative of $\cos(x)$:

$$\frac{d}{dx} \cos(x) = -\sin(x)$$

2. Integrate both sides with respect to $x$:

$$\int \frac{d}{dx} \cos(x) \, dx = \int -\sin(x) \, dx$$

3. Use t...
====================================================================
💻 Expert: CODE
Query:  Write a recursive function to compute Fibonacci numbers in Python.
Answer: **Fibonacci Recursive Function in Python**
...
====================================================

The Transformer model, introduced in the paper "Attention Is All You Need" by Vaswani et al. in 2017, revolutionized the field of natural language processing (NLP) by using self-attent
====================================================================
```

```
Query: 'Explain recursion.'
====================================================================

💻 CODE Expert:
**Recursion in Programming**
================================

Recursion is a fundamental concept in programming where a function calls itself repeatedly until it reaches a base case that stops the recursion. This technique allows us to break down
--------------------------------------------------------------------

✍️ CREATIVE Expert:
In the depths of a mystical forest, there existed a labyrinthine temple, shrouded in mystery. The temple's entrance was guarded by a wise and enigmatic sorceress, who whispered secrets
--------------------------------------------------------------------

🔬 SCIENCE Expert:
Recursion is a fundamental concept in computer science and mathematics that can be a bit tricky to grasp at first, but don't worry, I'm here to help you understand it with some analogi

**What is Recursion?**

Recursion is a programming technique where a function calls its...
--------------------------------------------------------------------
```

Summary

Generation Enhancement — Full Picture

Technique	What It Does	When to Use	Key Library
Full Fine-Tuning	Updates all weights	Large budget, very specialized domain	`transformers`, `Trainer`
LoRA / PEFT	Trains tiny adapter matrices (< 1% of params)	Medium budget, task adaptation	`peft`
FP16 / BF16	Half-precision training/inference	Always — halves memory, negligible quality loss	`torch.float16`
INT8 Quantization	8-bit weights (4x memory reduction)	Inference on consumer hardware	`bitsandbytes`
INT4 / GPTQ	4-bit weights (8x memory reduction)	Running 70B models on a single GPU	`bitsandbytes`, `auto-gptq`
Software MoE	Routes queries to specialist prompts	Multi-domain chatbots, enterprise assistants	LangChain
Model MoE	Sparse expert activation per token	Best quality-per-FLOP at scale	Mixtral, GPT-4

LoRA Key Formula

$$W' = W + \Delta W = W + BA \quad \text{where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}, r \ll d$$

Quantization Key Formula

$$x_{\text{INT8}} = \text{round}\left(\frac{x}{S}\right) + Z, \quad S = \frac{x_{\max} - x_{\min}}{255}$$

MoE Key Concept

$$\text{output} = \sum_{i \in \text{Top-K}} G(x)_i \cdot E_i(x)$$

Where $G(x)$ is the router's gating probability and $E_i(x)$ is the output of expert i.